药品管理模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 药品实体表(medicine)

```sql
CREATE TABLE `medicine` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '药品ID',
  `medicine_code` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NOT NULL COMMENT '药品编码',
  `medicine_name` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NOT NULL COMMENT '药品名称',
  `specification` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NULL DEFAULT NULL COMMENT '规格',
  `dosage_form` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '剂型(片剂/胶囊/注射剂等)',
  `manufacturer` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NULL DEFAULT NULL COMMENT '生产厂家',
  `category` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
DEFAULT NULL COMMENT '类别(处方药/非处方药)',
  `category_id` bigint NULL DEFAULT NULL COMMENT '分类ID',
  `price` decimal(10, 2) NULL DEFAULT NULL COMMENT '单价',
  `stock` int NULL DEFAULT 0 COMMENT '库存量',
  `instructions` text CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL
COMMENT '使用说明',
  `status` tinyint NULL DEFAULT 1 COMMENT '状态(0:下架,1:上架)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `uk_medicine_code`(`medicine_code` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 1 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '药品表' ROW_FORMAT = Dynamic;
```

Medicine实体类:

```java
@Data
@TableName("medicine")
@Schema(description = "药品实体类")
public class Medicine {
    @TableId(type = IdType.AUTO)
    @Schema(description = "药品ID")
    private Long id;

    @Schema(description = "药品编码")
    private String medicineCode;

    @Schema(description = "药品名称")
    private String medicineName;

    @Schema(description = "规格")
```

半夜撕代码版权所有

```
    private String specification;

    @Schema(description = "剂型")
    private String dosageForm;

    @Schema(description = "生产厂家")
    private String manufacturer;

    @Schema(description = "类别(处方药/非处方药)")
    private String category;

    @Schema(description = "分类ID")
    private Long categoryId;

    @Schema(description = "单价")
    private BigDecimal price;

    @Schema(description = "库存量")
    private Integer stock;

    @Schema(description = "状态(0:下架,1:上架)")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @Schema(description = "使用说明")
    private String instructions;
}
```

1.1.2 药品分类实体表(medicine_category)

```
CREATE TABLE `medicine_category` (
  `id` bigint NOT NULL AUTO_INCREMENT COMMENT '分类ID',
  `category_name` varchar(50) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NOT NULL COMMENT '分类名称',
  `category_code` varchar(20) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NOT NULL COMMENT '分类编码',
  `description` varchar(200) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
NULL DEFAULT NULL COMMENT '分类描述',
  `status` tinyint NULL DEFAULT 1 COMMENT '状态(0:禁用,1:启用)',
  `create_time` datetime NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
  `update_time` datetime NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP COMMENT '更新时间',
  PRIMARY KEY (`id`) USING BTREE,
  UNIQUE INDEX `uk_category_code`(`category_code` ASC) USING BTREE
) ENGINE = InnoDB AUTO_INCREMENT = 8 CHARACTER SET = utf8mb4 COLLATE =
utf8mb4_0900_ai_ci COMMENT = '药品分类表' ROW_FORMAT = DYNAMIC;
```

MedicineCategory实体类:

半夜撕代码版权所有

```java
@Data
@TableName("medicine_category")
@Schema(description = "药品分类实体类")
public class MedicineCategory {
    @TableId(type = IdType.AUTO)
    @Schema(description = "分类ID")
    private Long id;

    @Schema(description = "分类名称")
    private String categoryName;

    @Schema(description = "分类编码")
    private String categoryCode;

    @Schema(description = "分类描述")
    private String description;

    @Schema(description = "状态：0-禁用，1-启用")
    private Integer status;

    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;
}
```

1.2 核心属性说明

1.2.1 药品实体核心属性

- **id**: 药品的唯一标识符，自增长
- **medicineCode**: 药品编码，唯一标识药品的业务代码，如"M001"
- **medicineName**: 药品名称，如"阿莫西林胶囊"
- **specification**: 药品规格，如"0.5g*10片"
- **dosageForm**: 剂型，如片剂、胶囊、注射剂等
- **manufacturer**: 生产厂家，如"华北制药"
- **category**: 药品类别，如"处方药"或"非处方药"
- **categoryId**: 关联药品分类表的ID
- **price**: 药品单价，精确到两位小数
- **stock**: 库存量，整数
- **instructions**: 使用说明，文本格式
- **status**: 状态，1表示上架，0表示下架

1.2.2 药品分类实体核心属性

- **id**: 分类的唯一标识符，自增长
- **categoryName**: 分类名称，如"抗生素"
- **categoryCode**: 分类编码，唯一标识分类的业务代码，如"ANTIBIOTIC"

半夜撕代码版权所有

- **description**: 分类描述，文本格式
- **status**: 状态，1表示启用，0表示禁用

二、各功能详细讲解

2.1 药品基础管理功能

2.1.1 功能概述

药品基础管理功能允许医院管理人员对药品信息进行全面管理，包括新增、编辑、查询、删除药品，以及对药品状态和库存进行管理。系统支持按多种条件搜索药品，如药品名称、编码、分类等，并提供分页查询功能，方便管理大量药品数据。

2.1.2 详细讲解功能实现流程

1. **药品列表查询流程**：
   前端在vue3/src/views/backend/medicine/index.vue中，通过onMounted钩子调用fetchMedicines方法，该方法使用request.get向/medicine/page接口发送请求，传递搜索条件和分页参数。后端MedicineController的getMedicinesByPage方法接收请求，调用medicineService.getMedicinesByPage方法进行查询，该方法使用MyBatis-Plus的LambdaQueryWrapper构建动态查询条件，最后将查询结果返回给前端。
2. **新增药品流程**：
   前端点击"新增药品"按钮，弹出对话框，填写药品信息后，点击提交按钮，调用submitForm方法，该方法通过request.post向/medicine接口发送请求。后端MedicineController的createMedicine方法接收请求，调用medicineService.createMedicine方法，该方法首先检查药品编码是否已存在，然后同步分类信息，最后将药品信息保存到数据库。
3. **编辑药品流程**：
   前端点击药品列表中的"编辑"按钮，弹出对话框，修改药品信息后，点击提交按钮，调用submitForm方法，该方法通过request.put向/medicine/{id}接口发送请求。后端MedicineController的updateMedicine方法接收请求，调用medicineService.updateMedicine方法，该方法首先检查药品是否存在，然后检查药品编码是否与其他药品冲突，最后更新药品信息。
4. **删除药品流程**：
   前端点击药品列表中的"删除"按钮，弹出确认对话框，点击确认后，调用handleDelete方法，该方法通过request.delete向/medicine/{id}接口发送请求。后端MedicineController的deleteMedicine方法接收请求，调用medicineService.deleteMedicine方法，该方法首先检查药品是否存在，然后检查药品是否被用于处方明细，如果没有则删除药品。
5. **药品库存调整流程**：
   前端在药品详情对话框中，输入库存调整数量，点击"确认调整"按钮，调用handleStockAdjust方法，该方法通过request.put向/medicine/{id}/stock接口发送请求。后端MedicineController的updateStock方法接收请求，调用medicineService.updateStock方法，该方法首先获取药品信息，然后计算新库存，检查库存是否充足，最后更新药品库存。

2.1.3 关键代码讲解

1. **药品创建代码**
   【springboot/src/main/java/org/example/springboot/service/MedicineService.java：createMedicine方法】

```
@Transactional
public Medicine createMedicine(Medicine medicine) {
    // 检查药品编码是否已存在
```

半夜撕代码版权所有

```
    if (medicineMapper.selectOne(
            new LambdaQueryWrapper<Medicine>()
                .eq(Medicine::getMedicineCode, medicine.getMedicineCode())
        ) != null) {
        throw new ServiceException("药品编码已存在");
    }

    // 确保category和categoryId同步
    syncCategoryInfo(medicine);

    // 设置创建时间和更新时间
    LocalDateTime now = LocalDateTime.now();
    medicine.setCreateTime(now);
    medicine.setUpdateTime(now);

    // 设置默认状态
    if (medicine.getStatus() == null) {
        medicine.setStatus(1); // 默认上架
    }

    if (medicineMapper.insert(medicine) <= 0) {
        throw new ServiceException("药品添加失败");
    }

    return medicine;
}
```

这段代码实现了创建药品的业务逻辑。首先检查药品编码是否已存在，避免重复编码。然后调用syncCategoryInfo方法同步药品分类信息，确保category和categoryId保持一致。接着设置创建时间、更新时间和默认状态，最后将药品信息插入数据库。

2. **药品分类同步代码**
【springboot/src/main/java/org/example/springboot/service/MedicineService.java：syncCategoryInfo方法】

```
private void syncCategoryInfo(Medicine medicine) {
    // 如果设置了categoryId但没有设置category
    if (medicine.getCategoryId() != null &&
StringUtils.isBlank(medicine.getCategory())) {
        MedicineCategory category =
medicineCategoryService.getById(medicine.getCategoryId());
        if (category != null) {
            medicine.setCategory(category.getCategoryName());
        }
    }
    // 如果设置了category但没有设置categoryId
    else if (StringUtils.isNotBlank(medicine.getCategory()) &&
medicine.getCategoryId() == null) {
        List<MedicineCategory> categories =
medicineCategoryService.getAllCategories();
        for (MedicineCategory category : categories) {
            if (category.getCategoryName().equals(medicine.getCategory())) {
                medicine.setCategoryId(category.getId());
                break;
            }
```

半夜撕代码版权所有

```
        }
    }
    // 两者都没设置时，设置默认值
    else if (StringUtils.isBlank(medicine.getCategory()) &&
medicine.getCategoryId() == null) {
        medicine.setCategory("未分类");
    }
}
```

这段代码实现了同步药品分类信息的逻辑。它处理三种情况：1) 如果只设置了categoryId，则根据categoryId查询对应的分类名称，并设置到category字段；2) 如果只设置了category，则查询所有分类，找到匹配的分类名称，并设置对应的categoryId；3) 如果两者都没设置，则设置默认分类名称为"未分类"。这样确保了药品的分类信息的一致性。

3. **药品库存更新代码**
【springboot/src/main/java/org/example/springboot/service/MedicineService.java：updateStock方法】

```
@Transactional
public void updateStock(Long id, Integer stockChange) {
    Medicine medicine = getMedicineById(id);

    // 计算新库存
    int newStock = medicine.getStock() + stockChange;
    if (newStock < 0) {
        throw new ServiceException("库存不足");
    }

    Medicine updateMedicine = new Medicine();
    updateMedicine.setId(id);
    updateMedicine.setStock(newStock);
    updateMedicine.setUpdateTime(LocalDateTime.now());

    if (medicineMapper.updateById(updateMedicine) <= 0) {
        throw new ServiceException("库存更新失败");
    }
}
```

这段代码实现了更新药品库存的业务逻辑。它首先获取药品信息，然后根据传入的stockChange计算新库存。如果新库存小于0，则抛出"库存不足"的异常，否则更新药品的库存和更新时间。这个方法支持药品入库（stockChange为正数）和出库（stockChange为负数）操作。

4. **药品删除前检查代码**
【springboot/src/main/java/org/example/springboot/service/MedicineService.java：deleteMedicine方法】

```
@Transactional
public void deleteMedicine(Long id) {
    // 检查药品是否存在
    Medicine medicine = medicineMapper.selectById(id);
    if (medicine == null) {
        throw new ServiceException("药品不存在");
    }

    // 检查药品是否被用于处方明细
```

半夜撕代码版权所有

```
    LambdaQueryWrapper<PrescriptionDetail> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(PrescriptionDetail::getMedicineId, id);
    long count = prescriptionDetailMapper.selectCount(queryWrapper);
    if (count > 0) {
        throw new ServiceException("该药品已被用于处方，不能删除");
    }

    if (medicineMapper.deleteById(id) <= 0) {
        throw new ServiceException("药品删除失败");
    }
}
```

这段代码实现了删除药品前的安全检查逻辑。它首先检查药品是否存在，然后查询处方明细表，检查药品是否被用于处方。如果药品已被用于处方，则不允许删除，抛出相应的异常。这样可以避免删除已经使用过的药品，保证数据的完整性和一致性。

2.1.4 功能实现细节

1. **药品编码唯一性检查**：系统在创建和更新药品时，会检查药品编码的唯一性，避免重复编码导致的数据混乱。
2. **药品分类信息同步**：系统实现了药品分类信息的自动同步，确保category和categoryId字段保持一致，提高数据质量。
3. **药品安全删除检查**：系统在删除药品前，会检查药品是否被用于处方明细，避免删除已使用的药品导致的数据不一致。
4. **药品库存安全检查**：系统在更新药品库存时，会检查新库存是否小于0，避免出现负库存的情况。

2.2 药品分类管理功能

2.2.1 功能概述

药品分类管理功能允许医院管理人员对药品分类进行管理，包括新增、编辑、查询、删除分类。系统支持按分类名称、编码、状态等条件搜索分类，并提供分页查询功能。药品分类是药品管理的基础，可以帮助医院更好地组织和管理药品。

2.2.2 详细讲解功能实现流程

1. **分类列表查询流程**：
   前端在vue3/src/views/backend/medicine-category/index.vue中，通过onMounted钩子调用fetchCategories方法，该方法使用request.get向/medicine-category/page接口发送请求，传递搜索条件和分页参数。后端MedicineCategoryController的getCategoriesByPage方法接收请求，调用medicineCategoryService.getCategoriesByPage方法进行查询，该方法使用MyBatis-Plus的LambdaQueryWrapper构建动态查询条件，最后将查询结果返回给前端。
2. **新增分类流程**：
   前端点击"新增分类"按钮，弹出对话框，填写分类信息后，点击提交按钮，调用submitForm方法，该方法通过request.post向/medicine-category接口发送请求。后端MedicineCategoryController的createCategory方法接收请求，调用medicineCategoryService.add方法，将分类信息保存到数据库。
3. **编辑分类流程**：
   前端点击分类列表中的"编辑"按钮，弹出对话框，修改分类信息后，点击提交按钮，调用submitForm方法，该方法通过request.put向/medicine-category/{id}接口发送请求。后端MedicineCategoryController的updateCategory方法接收请求，调用

半夜撕代码版权所有

medicineCategoryService.update方法，更新分类信息。

4. **删除分类流程**：
   前端点击分类列表中的"删除"按钮，弹出确认对话框，点击确认后，调用handleDelete方法，该方法通过request.delete向/medicine-category/{id}接口发送请求。后端MedicineCategoryController的deleteCategory方法接收请求，调用medicineCategoryService.delete方法，删除分类。

2.2.3 关键代码讲解

1. **分类查询代码**
   【springboot/src/main/java/org/example/springboot/service/MedicineCategoryService.java：getCategoriesByPage方法】

```
public Page<MedicineCategory> getCategoriesByPage(String categoryName, String
categoryCode, Integer status,
                                    Integer currentPage, Integer size) {
    LambdaQueryWrapper<MedicineCategory> queryWrapper = new
LambdaQueryWrapper<>();

    // 添加查询条件
    if (StringUtils.isNotBlank(categoryName)) {
        queryWrapper.like(MedicineCategory::getCategoryName, categoryName);
    }
    if (StringUtils.isNotBlank(categoryCode)) {
        queryWrapper.like(MedicineCategory::getCategoryCode, categoryCode);
    }
    if (status != null) {
        queryWrapper.eq(MedicineCategory::getStatus, status);
    }

    // 按创建时间降序排序
    queryWrapper.orderByDesc(MedicineCategory::getCreateTime);

    return medicineCategoryMapper.selectPage(new Page<>(currentPage, size),
queryWrapper);
}
```

   这段代码实现了分页查询药品分类的业务逻辑。它根据传入的参数构建动态查询条件，支持按分类名称、分类编码和状态进行模糊查询或精确匹配，并按创建时间降序排序，最后返回分页结果。

2. **分类编码检查代码**
   【springboot/src/main/java/org/example/springboot/service/MedicineCategoryService.java：checkCategoryCodeExists方法】

半夜撕代码版权所有

```
public boolean checkCategoryCodeExists(String categoryCode, Long excludeId) {
    LambdaQueryWrapper<MedicineCategory> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(MedicineCategory::getCategoryCode, categoryCode);

    if (excludeId != null) {
        queryWrapper.ne(MedicineCategory::getId, excludeId);
    }

    return medicineCategoryMapper.selectCount(queryWrapper) > 0;
}
```

这段代码实现了检查分类编码是否已存在的业务逻辑。它根据传入的分类编码构建查询条件，如果传入了excludeId参数，则排除该ID的分类，最后返回是否存在符合条件的分类。这个方法通常用于创建和更新分类时，检查分类编码的唯一性。

2.2.4 功能实现细节

1. **分类编码格式验证**：前端在提交分类表单时，会验证分类编码的格式，确保只包含大写字母和数字。
2. **分类状态管理**：系统支持启用和禁用分类，方便管理员控制分类的使用状态。
3. **分类信息展示**：前端列表展示分类的基本信息，包括编码、名称、描述、状态和创建时间，方便管理员查看和管理。

2.3 药品库存管理功能

2.3.1 功能概述

药品库存管理功能允许医院管理人员对药品库存进行管理，包括查看库存、调整库存（入库和出库）。系统提供了直观的库存展示和调整界面，方便管理员实时掌握药品库存情况，并进行库存调整操作。

2.3.2 详细讲解功能实现流程

1. **查看药品库存流程**：
   前端在药品列表中显示每个药品的当前库存量，点击"详情"按钮，可以查看药品的详细信息，包括库存量。后端通过MedicineController的getMedicineById方法获取药品详情，包括库存信息。
2. **调整药品库存流程**：
   前端在药品详情对话框中，输入库存调整数量（正数表示入库，负数表示出库），点击"确认调整"按钮，调用handleStockAdjust方法，该方法通过request.put向/medicine/{id}/stock接口发送请求。后端MedicineController的updateStock方法接收请求，调用medicineService.updateStock方法，更新药品库存。

2.3.3 关键代码讲解

1. **前端库存调整代码**
   【vue3/src/views/backend/medicine/index.vue：handleStockAdjust方法】

```
const handleStockAdjust = async () => {
  if (stockChange.value === 0) {
    ElMessage.warning('请输入调整数量')
    return
  }
```

半夜撕代码版权所有

```
  try {
    await request.put(`/medicine/${detailForm.id}/stock?
stockChange=${stockChange.value}`, {

    }, {
      successMsg: '库存调整成功',
      onSuccess: () => {
        // 更新详情中的库存数量
        detailForm.stock += stockChange.value
        stockChange.value = 0

        // 刷新列表
        fetchMedicines()
      }
    })
  } catch (error) {
    console.error('库存调整失败:', error)
  }
}
```

这段代码实现了前端调整药品库存的逻辑。它首先检查调整数量是否为0，如果是则提示用户输入调整数量。然后发送PUT请求到后端接口，传递药品ID和库存变化量。请求成功后，更新详情对话框中的库存数量，并刷新药品列表。

2. **后端库存调整代码**
【springboot/src/main/java/org/example/springboot/controller/MedicineController.java：updateStock方法】

```
@Operation(summary = "更新药品库存")
@PutMapping("/{id}/stock")
public Result<?> updateStock(@PathVariable Long id, @RequestParam Integer
stockChange) {
    LOGGER.info("更新药品库存: id={}, stockChange={}", id, stockChange);
    medicineService.updateStock(id, stockChange);
    return Result.success();
}
```

这段代码是后端控制器中的库存调整接口。它接收药品ID和库存变化量作为参数，记录日志后调用medicineService.updateStock方法执行库存调整操作，最后返回成功结果。

2.3.4 功能实现细节

1. **库存安全检查**：系统在调整库存时，会检查出库操作是否会导致库存为负，如果是则拒绝操作并提示"库存不足"。
2. **库存变化记录**：系统在调整库存时，会更新药品的更新时间，方便追踪库存变化的时间点。
3. **库存调整提示**：前端在库存调整界面，提示用户正数表示入库，负数表示出库，方便用户理解操作含义。

半夜撕代码版权所有

2.4 药品与处方的关联功能

2.4.1 功能概述

药品与处方的关联功能实现了药品在处方中的使用。当医生开具处方时，可以选择药品并指定用量、频次等信息，系统会记录药品的使用情况，并在删除药品前检查是否被用于处方，避免删除已使用的药品。

2.4.2 详细讲解功能实现流程

1. **处方中使用药品流程**：
   医生在开具处方时，可以从药品列表中选择药品，并指定用量、频次、用药天数等信息。系统会将这些信息保存到处方明细表中，建立处方与药品的关联关系。
2. **药品删除前检查流程**：
   当管理员尝试删除药品时，系统会检查该药品是否被用于处方明细。如果已被使用，则不允许删除，避免数据不一致。

2.4.3 关键代码讲解

1. **药品删除前检查代码**
   【springboot/src/main/java/org/example/springboot/service/MedicineService.java：deleteMedicine方法】

```
@Transactional
public void deleteMedicine(Long id) {
    // 检查药品是否存在
    Medicine medicine = medicineMapper.selectById(id);
    if (medicine == null) {
        throw new ServiceException("药品不存在");
    }

    // 检查药品是否被用于处方明细
    LambdaQueryWrapper<PrescriptionDetail> queryWrapper = new
LambdaQueryWrapper<>();
    query
```

我是Claude 3.7 Sonnet模型。我将继续编写药品管理模块的讲解文档。

Read file: springboot/src/main/java/org/example/springboot/controller/MedicineCategoryController.java
继续编写药品管理模块的讲解文档：

2.4.3 关键代码讲解（续）

1. **药品删除前检查代码**
   【springboot/src/main/java/org/example/springboot/service/MedicineService.java：deleteMedicine方法】

```
@Transactional
public void deleteMedicine(Long id) {
    // 检查药品是否存在
    Medicine medicine = medicineMapper.selectById(id);
    if (medicine == null) {
        throw new ServiceException("药品不存在");
```

半夜撕代码版权所有

```
    }

    // 检查药品是否被用于处方明细
    LambdaQueryWrapper<PrescriptionDetail> queryWrapper = new
LambdaQueryWrapper<>();
    queryWrapper.eq(PrescriptionDetail::getMedicineId, id);
    long count = prescriptionDetailMapper.selectCount(queryWrapper);
    if (count > 0) {
        throw new ServiceException("该药品已被用于处方，不能删除");
    }

    if (medicineMapper.deleteById(id) <= 0) {
        throw new ServiceException("药品删除失败");
    }
}
```

这段代码实现了删除药品前的安全检查逻辑。它首先检查药品是否存在，然后查询处方明细表，检查药品是否被用于处方。如果药品已被用于处方，则不允许删除，抛出相应的异常。这种设计可以保护数据完整性，避免删除已经在处方中使用的药品导致的数据不一致问题。

2. **处方明细与药品关联代码**
【springboot/src/main/java/org/example/springboot/entity/PrescriptionDetail.java】

```
@Data
@TableName("prescription_detail")
@Schema(description = "处方明细实体类")
public class PrescriptionDetail {
    @TableId(type = IdType.AUTO)
    @Schema(description = "明细ID")
    private Long id;

    @Schema(description = "处方ID")
    private Long prescriptionId;

    @Schema(description = "药品ID")
    private Long medicineId;

    @Schema(description = "用量")
    private String dosage;

    @Schema(description = "频次(一日三次/每日一次)")
    private String frequency;

    @Schema(description = "用药天数")
    private Integer days;

    @Schema(description = "数量")
    private Integer quantity;

    @Schema(description = "用法(口服/外用等)")
    @TableField("`usage`")
    private String usage;

    @Schema(description = "备注")
    private String notes;
```

半夜撕代码版权所有

```
    @Schema(description = "创建时间")
    private LocalDateTime createTime;

    @Schema(description = "更新时间")
    private LocalDateTime updateTime;

    @TableField(exist = false)
    @Schema(description = "药品信息")
    private Medicine medicine;
}
```

这段代码定义了处方明细实体类，它通过medicineId字段与药品建立关联关系。同时，它还包含了一个非数据库字段medicine，用于在查询处方明细时关联药品信息，方便前端展示完整的处方明细数据，包括药品名称、规格等信息。

2.4.4 功能实现细节

1. **药品关联检查**：系统在删除药品前，会检查药品是否被用于处方明细，如果是则不允许删除，保证数据一致性。
2. **药品信息关联**：在查询处方明细时，系统会自动关联药品信息，方便前端展示完整的处方明细数据。
3. **药品使用统计**：系统可以统计药品在处方中的使用情况，为医院药品管理提供数据支持。

2.5 药品分类管理功能

2.5.1 功能概述

药品分类管理功能允许医院管理人员对药品分类进行全面管理，包括新增、编辑、查询、删除分类。系统支持按分类名称、编码、状态等条件搜索分类，并提供分页查询功能。药品分类是药品管理的基础，可以帮助医院更好地组织和管理药品。

2.5.2 详细讲解功能实现流程

1. **分类列表查询流程**：
   前端在vue3/src/views/backend/medicine-category/index.vue中，通过onMounted钩子调用fetchCategories方法，该方法使用request.get向/medicine-category/page接口发送请求，传递搜索条件和分页参数。后端MedicineCategoryController的getCategoriesByPage方法接收请求，调用medicineCategoryService.getCategoriesByPage方法进行查询，最后将查询结果返回给前端。
2. **新增分类流程**：
   前端点击"新增分类"按钮，弹出对话框，填写分类信息后，点击提交按钮，调用submitForm方法，该方法通过request.post向/medicine-category接口发送请求。后端MedicineCategoryController的addCategory方法接收请求，首先检查分类名称和编码是否已存在，然后调用medicineCategoryService.add方法，将分类信息保存到数据库。
3. **编辑分类流程**：
   前端点击分类列表中的"编辑"按钮，弹出对话框，修改分类信息后，点击提交按钮，调用submitForm方法，该方法通过request.put向/medicine-category/{id}接口发送请求。后端MedicineCategoryController的updateCategory方法接收请求，首先检查分类是否存在，然后检查分类名称和编码是否与其他分类冲突，最后调用medicineCategoryService.update方法，更新分类信息。

半夜撕代码版权所有

4. **删除分类流程**：
前端点击分类列表中的"删除"按钮，弹出确认对话框，点击确认后，调用handleDelete方法，该方法通过request.delete向/medicine-category/{id}接口发送请求。后端MedicineCategoryController的deleteCategory方法接收请求，调用medicineCategoryService.delete方法，删除分类。

2.5.3 关键代码讲解

1. **分类名称和编码唯一性检查代码**
【springboot/src/main/java/org/example/springboot/controller/MedicineCategoryController.java：addCategory方法】

```
@Operation(summary = "新增药品分类")
@PostMapping
public Result<?> addCategory(@RequestBody MedicineCategory medicineCategory)
{
    LOGGER.info("新增药品分类: {}", medicineCategory.getCategoryName());

    // 检查分类名称是否已存在
    if
(medicineCategoryService.checkCategoryNameExists(medicineCategory.getCategory
Name(), null)) {
        return Result.error("分类名称已存在");
    }

    // 检查分类编码是否已存在
    if
(medicineCategoryService.checkCategoryCodeExists(medicineCategory.getCategory
Code(), null)) {
        return Result.error("分类编码已存在");
    }

    boolean success = medicineCategoryService.add(medicineCategory);
    return success ? Result.success(medicineCategory) : Result.error("新增失
败");
}
```

这段代码实现了新增药品分类时的唯一性检查逻辑。它首先检查分类名称是否已存在，然后检查分类编码是否已存在，如果存在则返回错误信息，否则调用服务层方法添加分类。这种设计可以确保分类名称和编码的唯一性，避免重复数据。

2. **分类查询代码**
【springboot/src/main/java/org/example/springboot/service/MedicineCategoryService.java：getAllCategories方法】

```
public List<MedicineCategory> getAllCategories() {
    return medicineCategoryMapper.selectList(
        new LambdaQueryWrapper<MedicineCategory>()
            .eq(MedicineCategory::getStatus, 1)
            .orderByAsc(MedicineCategory::getCategoryName)
    );
}
```

半夜撕代码版权所有

这段代码实现了获取所有启用状态的药品分类的业务逻辑。它使用MyBatis-Plus的LambdaQueryWrapper构建查询条件，只查询状态为启用（status=1）的分类，并按分类名称升序排序。这个方法通常用于在药品管理页面的下拉选择框中展示可用的分类选项。

2.5.4 功能实现细节

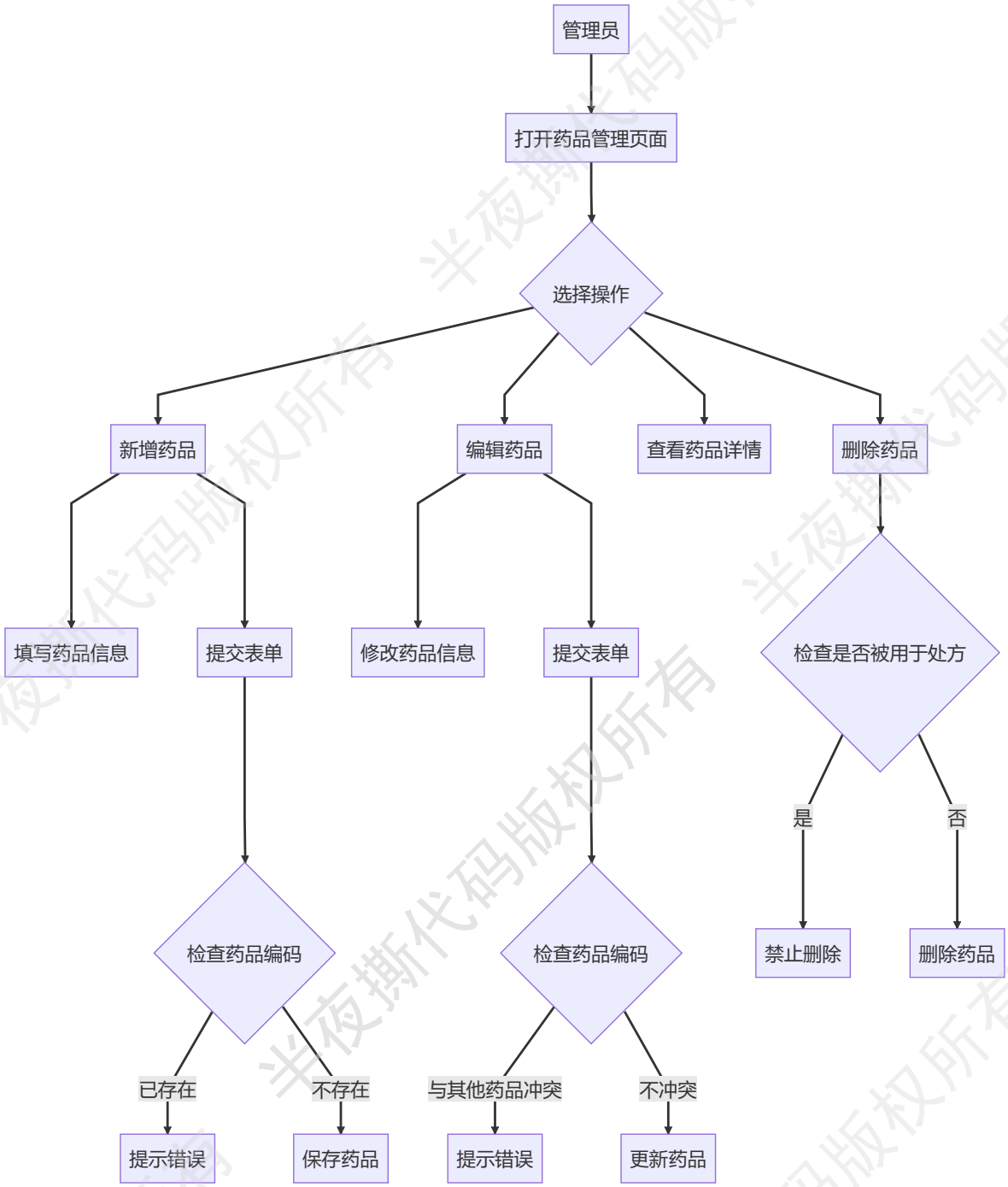
1. **分类名称和编码唯一性检查**：系统在创建和更新分类时，会检查分类名称和编码的唯一性，避免重复数据。
2. **分类状态管理**：系统支持启用和禁用分类，方便管理员控制分类的使用状态。
3. **分类编码格式验证**：前端在提交分类表单时，会验证分类编码的格式，确保只包含大写字母和数字。
4. **分类信息展示**：前端列表展示分类的基本信息，包括编码、名称、描述、状态和创建时间，方便管理员查看和管理。

三、总结

3.1 药品基础管理功能实现

药品基础管理功能通过MedicineController和MedicineService实现。管理员可以在后台管理页面创建、编辑、查询、删除药品，并对药品状态进行管理。系统支持按多种条件搜索药品，如药品名称、编码、分类等，并提供分页查询功能。在创建和更新药品时，系统会检查药品编码的唯一性，并同步药品分类信息，确保数据一致性。删除药品前，系统会检查药品是否被用于处方明细，避免删除已使用的药品导致的数据不一致。

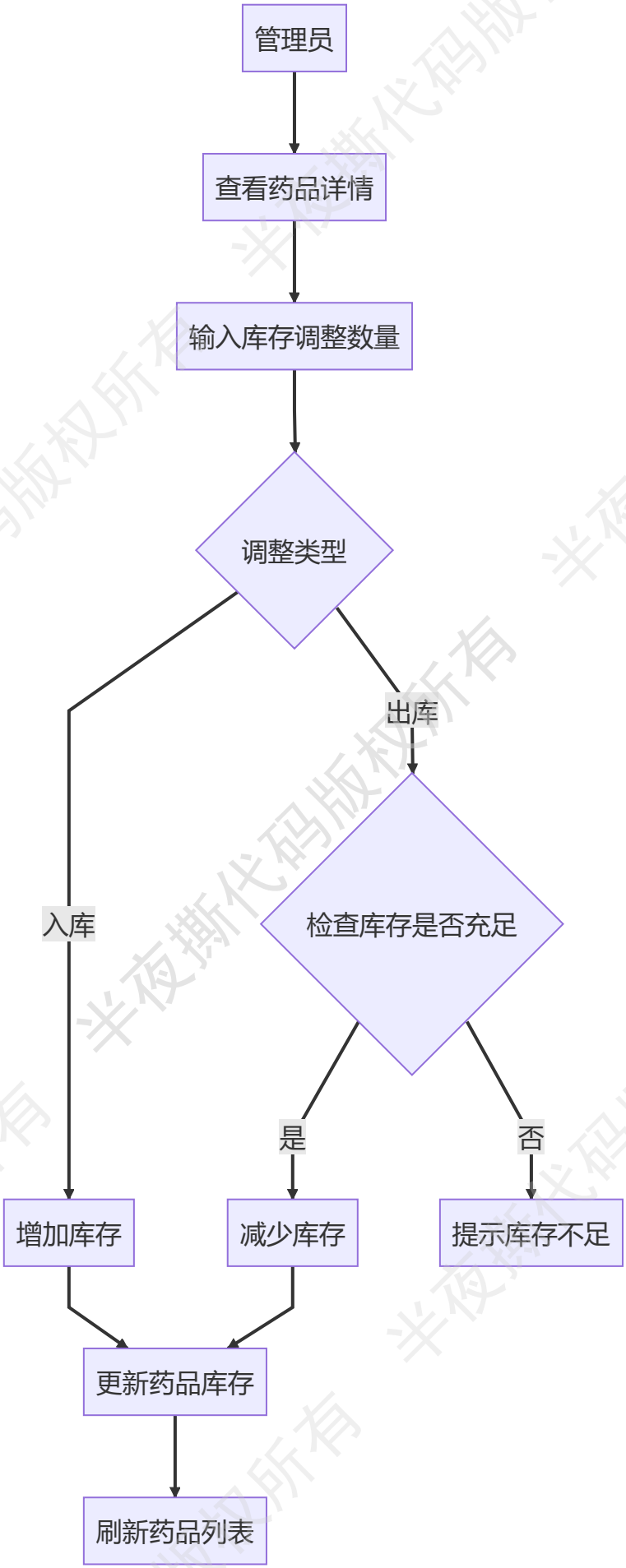
半夜撕代码版权所有



3.2 药品库存管理功能实现

药品库存管理功能通过MedicineController和MedicineService实现。管理员可以在药品详情页面查看库存并进行调整。系统支持入库（增加库存）和出库（减少库存）操作，并在出库时检查库存是否充足，避免出现负库存。库存调整操作会更新药品的库存量和更新时间，方便追踪库存变化。

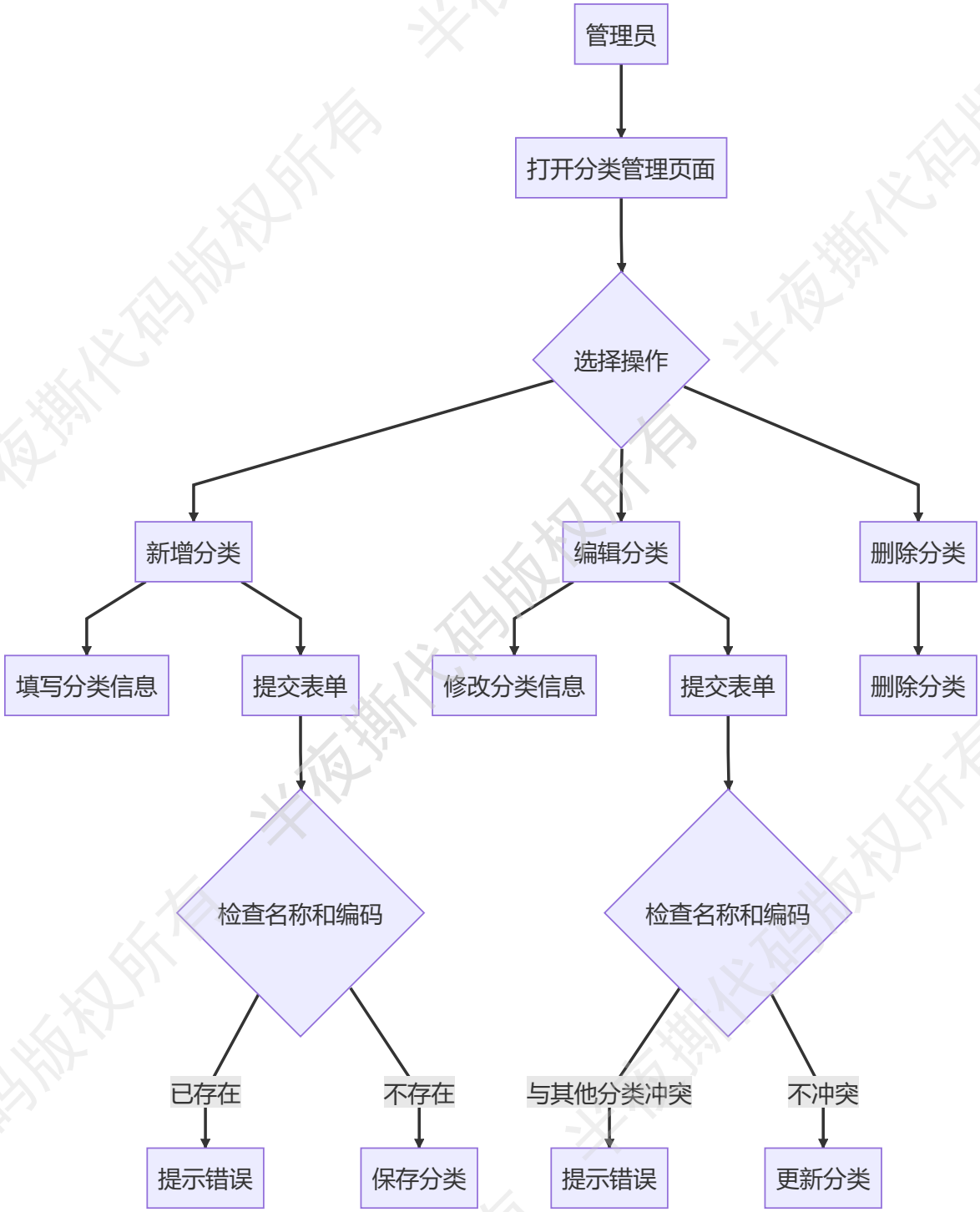
半夜撕代码版权所有


管理员
查看药品详情
输入库存调整数量
调整类型
入库
出库
检查库存是否充足
是
否
增加库存
减少库存
提示库存不足
更新药品库存
刷新药品列表

半夜撕代码版权所有

3.3 药品分类管理功能实现

药品分类管理功能通过MedicineCategoryController和MedicineCategoryService实现。管理员可以在后台管理页面创建、编辑、查询、删除分类。系统支持按分类名称、编码、状态等条件搜索分类，并提供分页查询功能。在创建和更新分类时，系统会检查分类名称和编码的唯一性，避免重复数据。分类可以设置为启用或禁用状态，方便管理员控制分类的使用。



3.4 药品与处方的关联功能实现

药品与处方的关联功能通过PrescriptionDetail实体类实现。处方明细表通过medicineId字段与药品建立关联关系，记录药品在处方中的使用情况，包括用量、频次、用药天数等信息。在删除药品前，系统会检查药品是否被用于处方明细，如果是则不允许删除，保证数据一致性。

半夜撕代码版权所有

药品管理模块是医院信息系统的重要组成部分，它实现了对药品的全生命周期管理，包括基础信息管理、库存管理、分类管理以及与处方的关联管理。通过这些功能，医院可以更好地管理药品资源，提高药品管理的效率和准确性，为患者提供更好的医疗服务。

半夜撕代码版权所有